

See discussions, stats, and author profiles for this publication at: <https://www.researchgate.net/publication/308401918>

Design Asynchronous Circuit Using the MTNCL Pipeline

Method · December 2014

DOI: 10.13140/RG.2.2.25779.43045

CITATIONS

0

READS

2,374

1 author:



[Liang Men](#)

Apple Inc.

13 PUBLICATIONS **42** CITATIONS

SEE PROFILE

Design Asynchronous Circuit using the MTNCL Pipeline

--Practice with the FIR Filter

Liang Men

mliang@uark.edu

1. MTNCL pipeline architecture

The MTNCL pipeline in Fig.1 is the structure used to build the MTNCL circuit. It looks similar to the Boolean pipeline. The registers are put at the input and output of the combinational logic to form one pipeline stage. One additional component is called Early Completion Detection (the *comp* in the pipeline). It is used to generate the handshaking signals, *Ki* and *Ko*. So there are three components in the pipeline, the register (*regm*), the combinational logic (*comb*) and the completion detection (*comp*).

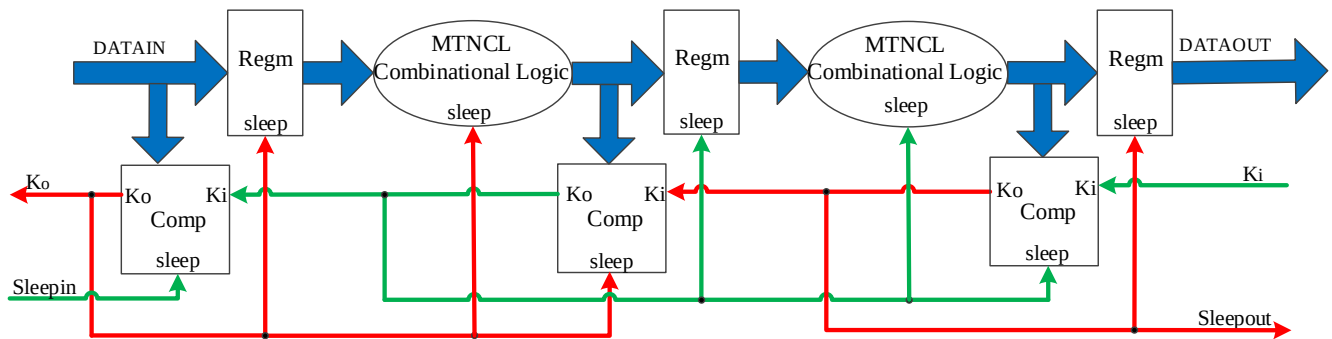
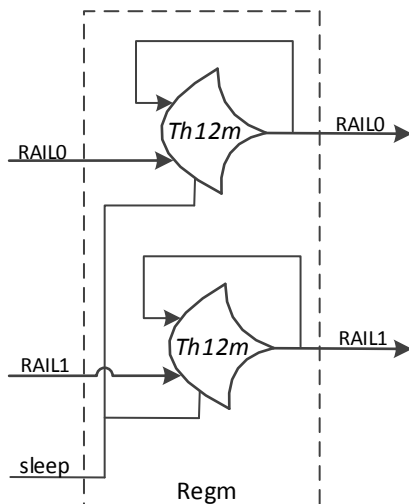


Fig.1. MTNCL Pipeline Architecture

Let's start with the components and then find out how the pipeline works with a real circuit. First it is the register, which has a simple structure as shown in Fig.2. The VHDL code is attached on the left. Basically the output is 'NULL' when sleep is '1', otherwise, the output has the same 'DATA' as the input.



```
Gr0: th12x0m
port map(a.rail0, t0, sleep, t0);
Gr1: th12x0m
port map(a.rail1, t1, sleep, t1);
z.rail0 <= t0;
z.rail1 <= t1;
```

Fig.2. MTNCL Register

One typical implementation of the completion detection logic is shown in Fig. 3. The Generic VHDL code for the completion detection block is attached below the figure. The completion detection logic is one key component related to the propagation delay of the pipeline. It has three stages. The first stage is the data detection, which can be built with th12m or th24comp0m gates, based on the number of input bits. If all the inputs are in DATA cycle, the outputs of the first stage will be all '1's. The second stage is the sleepable *ANDTREE*, which has a generic design with th44m, th33m and th22m gates based on the number of input bits. It merges all the inputs to one output signal. If the sleep is '1', the output is '0'; else if all the inputs are '1', the output is '1', otherwise, the output stays unchanged. The last stage is the 2-input C-element and an inverter. It inverts the Ki signal to generate the Ko signal. So the propagation delay of this component is not fixed for a DATA cycle. It depends on the number of input bits. The larger the number of input bits is, the longer the delay will be.

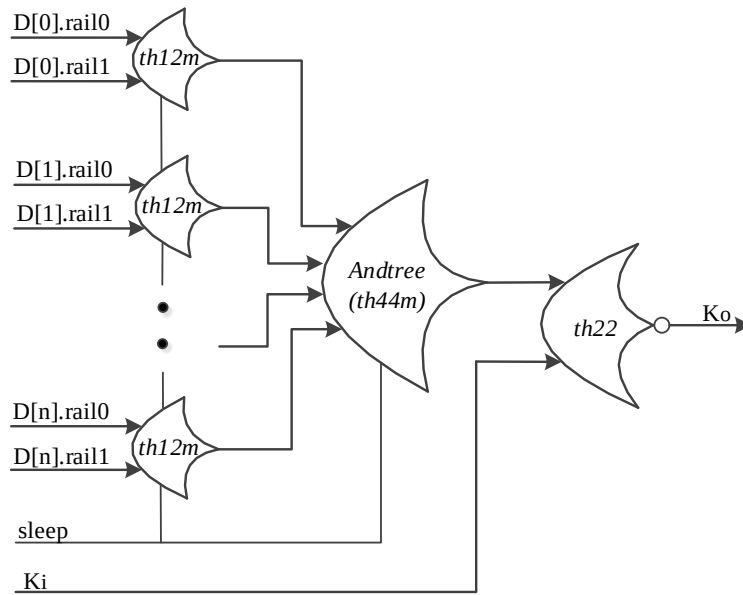


Fig.3. Completion Detection Block in MTNCL

```

STAGE1: for i in 0 to width/2-1 generate
    Gs1: th24comp0m
        port map(a(i*2).rail0,a(i*2).rail1, a(i*2+1).rail0, a(i*2+1).rail1, sleep,
            t(i));
end generate;
ONEMORE: if width rem 2 = 1 generate
    Gsom: th12x0m
        port map(a(width-1).rail0, a(width-1).rail1, sleep, t(width/2));
end generate;
Gcompodd: if width rem 2 = 1 generate
begin
    Gco: andtreem

```

```

        generic map(width/2+1)
        port map(t, sleep, tko);
    end generate;

Gcompeven: if width rem 2 = 0 generate
begin
Gce: andtreem
    generic map(width/2)
    port map(t(width/2-1 downto 0), sleep, tko);
end generate;

Gfgate: th22nx0
    port map(tko, ki, rst, ttko);

Gfinv: invx0
    port map(ttko, ko);
end arch;

```

The combination logic design in the MTNCL is easier than the NCL. For the MTNCL circuits generate the NULL wave in the pipeline, it doesn't have the two special design requirements as in the NCL. The MTNCL combinational logic can be directly mapped from the Sum of Product(SOP). Take the full adder as an example, after the derivation of the K-map from the truth table, the SOP of Cout and S are shown below:

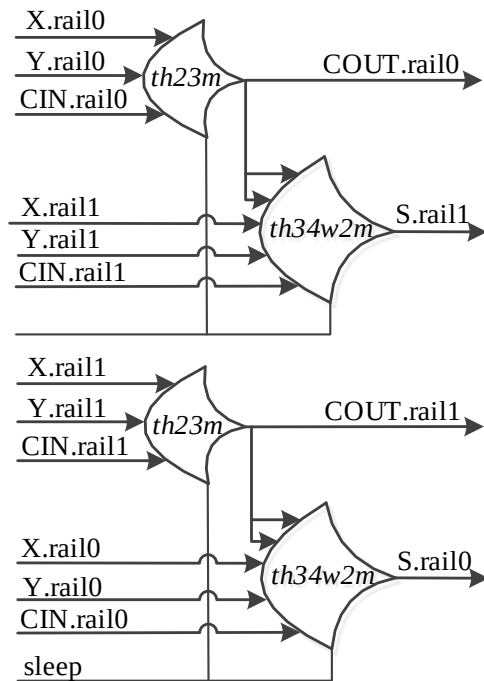
$$COUT^0 = X^0Y^0 + CIN^0X^0 + CIN^0Y^0$$

$$COUT^1 = X^1Y^1 + CIN^1X^1 + CIN^1Y^1$$

$$S^0 = X^0Y^0CIN^0 + X^0Y^1CIN^1 + X^1Y^0CIN^1 + X^1Y^1CIN^0$$

$$S^1 = X^0Y^0CIN^1 + X^0Y^1CIN^0 + X^1Y^0CIN^0 + X^1Y^1CIN^1$$

So Cout can be built with th23m gate and S can be built with th34w2m gate. The structure of the full adder is shown in Fig.4 and the VHDL implementation is on the left of the figure.



```

th0: th23x0m    port map
(X.rail0,Y.rail0,CIN.rail0,sleep,y0_buf
fer);

th1: th23x0m    port map
(X.rail1,Y.rail1,CIN.rail1,sleep,y1_buf
fer);

th2:th34w2x0m    port map
(y0_buffer,X.rail1,Y.rail1,CIN.rail1,s1
leep, s1_buffer);

th3:th34w2x0m    port map
(y1_buffer,X.rail0,Y.rail0,CIN.rail0,s1
leep,s0_buffer);--#

S.rail0 <= s0_buffer;
S.rail1 <= s1_buffer;
COUT.rail0 <= y0_buffer;
COUT.rail1 <= y1_buffer;

```

Fig.4. Full Adder in MTNCL

Now you know how the components in MTNCL work and how to build them with the MTNCL gates. By changing the combination logic in the pipeline, you can build a real circuit with the VHDL skills. The circuit will work, even though you don't know how it works. It sounds easy and we can take the generic adder as an example.

2. Generic Ripple Carry Adder

Building multiple input bits adder with the full adder and half adder is the fundamental knowledge in digital circuit design. There are also complex architectures like the carry-select and carry-look ahead adders to improve the throughput in the synchronous circuit. For the MTNCL implementation, we start with basic one, the ripple-carry adder.

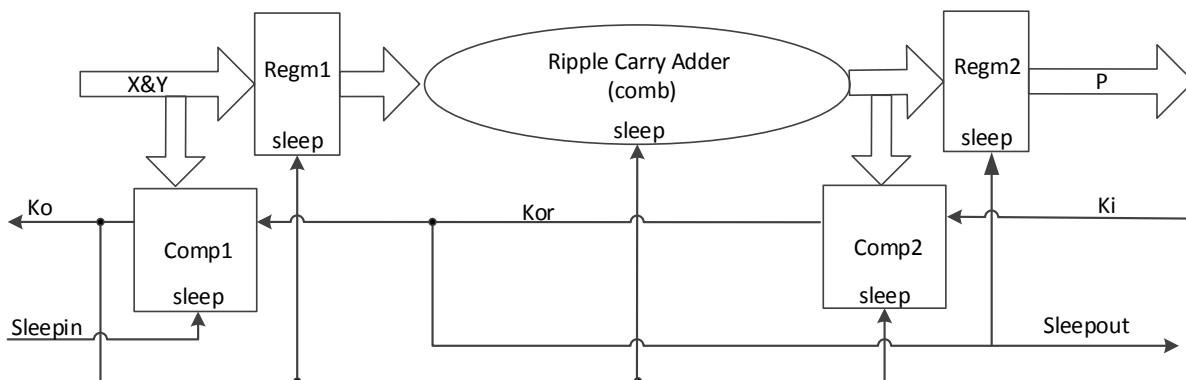


Fig.5. Ripple Carry Adder in MTNCL

Fig. 5 shows the design. After placing the regm and comp at the input and output of the combinational logic, which in the ripple carry adder in this case, it is done. The VHDL code for this design is attached in appendix I.

Now let's look at how the circuit works. After reset, all the handshaking signals are '1' and the internal data path are in 'NULL' state. Since Ko is '1' and is requesting for data (rfd), so you put a DATA cycle on the input path X and Y and change the sleepin to '0'. Then the DATA will be evaluated by the comp1 and the Ko will toggle to '0' after the evaluation time. Let's define the evaluation time as Tcomp_datain. Since Ko is connected to the sleep pins of the regm and comp, and those two components usually have large input capacitance, so you need a buffer to drive the sleep pins in comb and regm, as shown in Fig.6. It will add some propagation delay for the sleep signal. Let's define this propagation time as Tsleep_buf. So after Tsleep_buf, regm and comb 'wake up' (sleep = '0') and DATA can propagate through the register and be calculated by the ripple carry adder (comb). Let's define this time as the Tcomb. During this calculating time, the NULL cycle has already arrived at the input X and Y and sleepin is changed to '1'. But this NULL wave cannot be propagated until the output data from the comb got evaluated by the comp2 and Kor is changed to '0'. So for the NULL cycle, we need to add in the data evaluation time of Tcomp2. Let's call it Tcomp_dataout. Then the Ko can change to '1' and put the regm and comb to 'sleep', after the delay of Tsleep_buf. For the NULL wave generation, it is going to be fast in MTNCL. Let's ignore the delay for NULL wave generation to get a roughly estimation of the average DATA cycle in an MTNCL system, Tdd, which should be:

$$T_{dd} = T_{comp_datain} + 2 * T_{sleep_buf} + T_{comb} + T_{comp_dataout}$$

And the pipeline throughput would be

$$\text{Throughput} = 1/T_{dd}$$

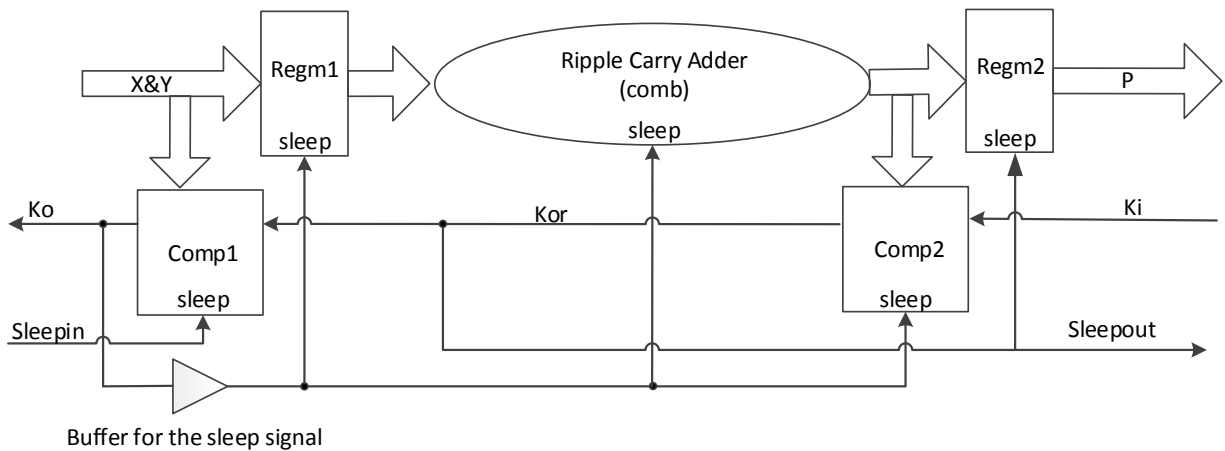


Fig.6. Ripple Carry Adder with Buffers for the Sleep Signal

So you can see that the propagation delay of the ripple carry adder (Tcomb) is only one of the three factors effecting the circuit throughput. This feature makes the MTNCL circuit different with the Boolean circuit. Some architecture in Boolean has a tradeoff between the hardware cost and throughput, like they can build the circuit really fast but very large. When you are trying to implement those architectures in MTNCL, take the delay of the buffer and comp into consideration. Otherwise, the throughput in the

asynchronous implementation could be worse. A typical example is the carry look-ahead adder implementation in NCL [1]. There is not much reference on comparing MTNCL implementation now, but don't be surprised if some fancy architectures in Boolean have worse throughput than the basic one in MTNCL.

So far, you know how the MTNCL pipeline works: the way that the DATA and NULL propagate through the components in the pipeline and the generation of the handshaking signals. And we also analyzed the main factors that dominating the T_{dd} in the pipeline. Then how to improve the throughput of the MTNCL pipeline? We know that the Boolean pipeline can be optimized for speed by partitioning the combinational circuitry and inserting additional registers. Does that work in the MTNCL pipeline? We will practice the method with another example.

3. Generic Carry-Save Multiplier

A 4x4 bit multiplier with Carry Save Adders (CSA) is shown in Fig.7 from reference [2]. The propagation delay for this multiplier is $8 \cdot \text{FA delay} + \text{AND delay}$. The CSA is combination of the Full Adder and an AND gate, but the AND gates are not on the critical path except the first CSA. Let's reduce the complexity and say the delay of this circuit is $8 \cdot \text{FA delay}$ and throughput is $1/(8 \cdot T_{fa})$, where T_{fa} is the propagation delay of the Full adder.

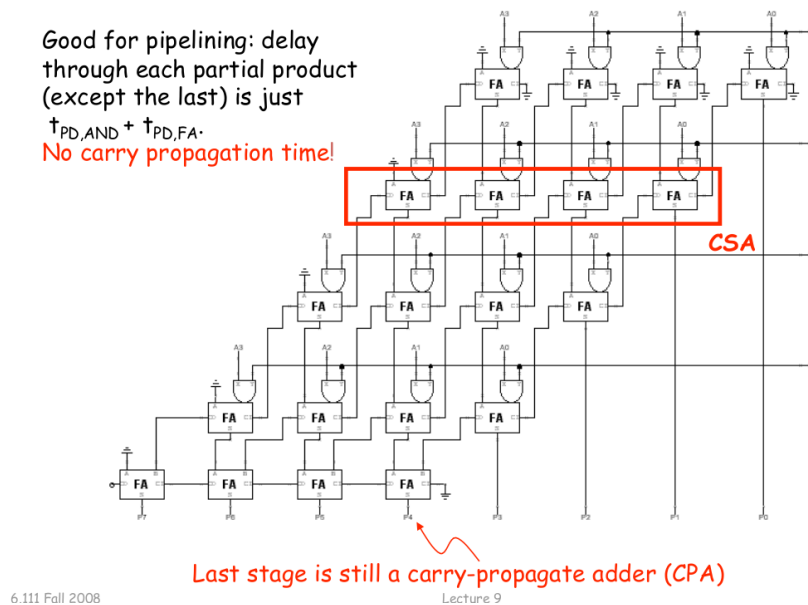


Fig.7. 4x4 bits Carry Save Multiplier [2]

Using this architecture, we can build the generic multiplier in MTNCL. Same to the generic adders, the implementation with the MTNCL pipeline is shown in Fig.8. The VHDL code is attached in appendix II.

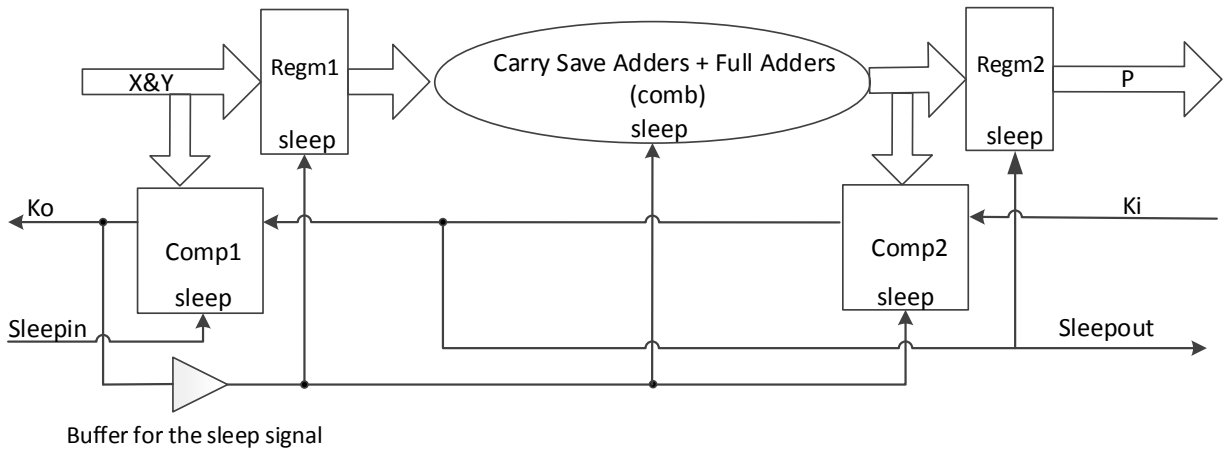


Fig.8. Generic Carry Save Multiplier in MTNCL

Now think about improving the throughput of the Fig.8 by adding in one more pipeline stage. For the Boolean design, inserting registers in the critical path to divide the propagation delay evenly will double the throughput, as shown in Fig.9. Will the result be the same with MTNCL pipeline? If we implement Fig.9 in MTNCL, we will end up with the circuit shown in Fig.10.

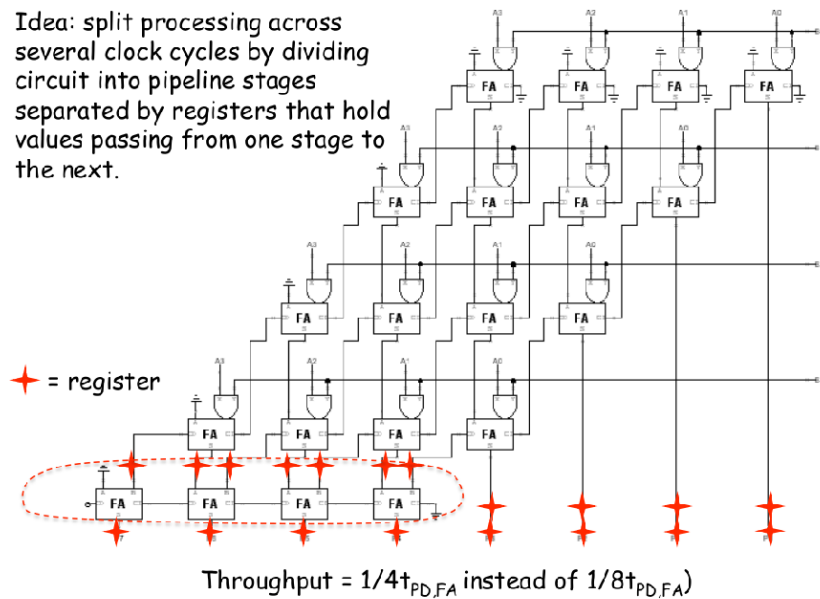


Fig.9. Double the Throughput of the Multiplier in Boolean [2]

In the design shown in Fig.10, Will T_{dd} be the same for the pipeline stage 1 and pipeline stage 2? Let's go back to the equation of the T_{dd} estimation. For the two pipeline stages, T_{comp_datain} , $T_{comp_dataout}$ and T_{comb} are the same. But the combination logic in stage 1 is much larger than the combination logic in stage 2. Then after adding buffers to the sleep signal, the T_{sleep_buf1} will be larger than T_{sleep_buf2} . The delay for the buffer is doubled in the T_{dd} equation, and the circuit throughput is based on the maximum T_{dd} in the pipeline stages. When the number of input bits scales up, the throughput of Fig. 10 will approach the throughput of Fig.8. So you can't design something like this in a generic style to double the throughput.

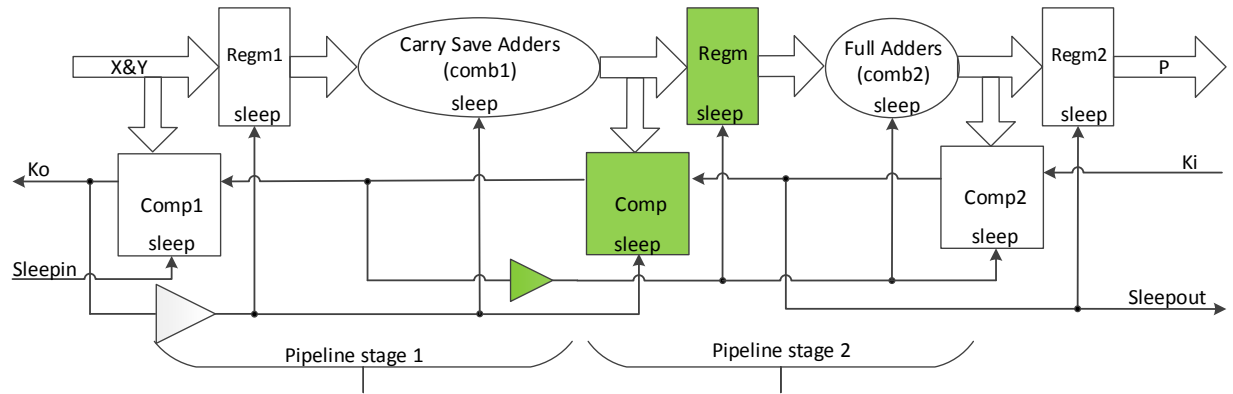


Fig.10. Carry Save Multiplier in MTNCL with Two Pipeline Stages

Partitioning the circuit and adding in more pipeline stages can improve the throughput, it is true in Boolean, NCL and MTNCL. For the Boolean, the method is inserting registers into the critical path and retiming for the balanced propagation delay. For the NCL, one more thing need to be considered is the number of the input/output bits. So there are bit-wise or full-word completion structures to reduce the delay for completion detection block. For the MTNCL, when you are trying to partition the combination logic, balancing the delay is equally important as balancing the input capacitance of the sleep pin. If the delay is balanced but the capacitance partition in bad, the sleep signal buffers will dominate T_{dd} and the throughput cannot be improved.

As a summary, we know that the throughput of the MTNCL circuit is the reciprocal of the maximum T_{dd} in the pipeline stages. And the T_{dd} is the combination of T_{comp} , T_{comb} and T_{sleep_buf} . So improving the throughput of the MTNCL circuit is design-dependent.

- (1) For the design with large propagation delay inside the combination logic, T_{comb} dominates T_{dd} . Partitioning the combinational circuitry and inserting additional register and completion components will be an effective way to improve throughput. It is the same with NCL and Boolean.
- (2) For the design with large number of input/output bits, T_{comp} dominates T_{dd} . Then try to reduce the delay of T_{comp} with bit-wise or full-word design in the completion detection block. The methodology is summarized in details in [3].
- (3) For the design with a large scale of the combination logic, T_{sleep_buf} dominates T_{dd} . Then inserting more pipeline stages and carefully designing the buffers for the sleep signal will be the way to improve throughput. It is a unique situation in MTNCL pipeline.

When you come to a real circuit, there is no design tools supporting the throughput optimization in MTNCL. So the optimization is usually carried out by analyzing the gate delay of each components and logic effort of the individual gates. There are a lot references on the NCL throughput optimization on [4]. And one research on analyzing the buffers for the sleep signal in MTNCL pipeline [5], but no example on the MTNCL throughput optimization so far.

4. 8-bit FIR filter with generic tap

Next we will design a more complex circuit. The FIR filter does the convolution operation for the input data. The structure is shown in Fig. 11.

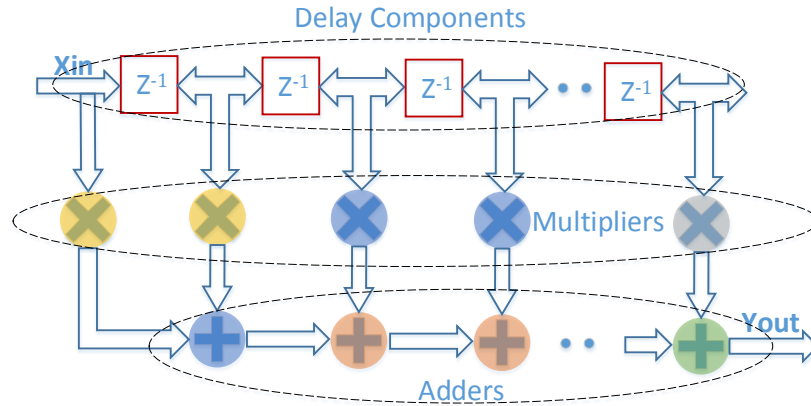


Fig.11. Hardware Structure of the FIR Filter

There are three components in the FIR circuit. The multiplier, the adder and delay component. We had the generic multiplier and generic adder in previous sections. If we choose the 8 by 8 bits multiplier for the FIR implementation, the combination logic delay for the multiplier is $16 \cdot T_{fa}$. Then we need to use the 16 by 16 bits adder for the first tap in the FIR, so the combination logic delay for the adder is the same with the multiplier. However, there are two issues with those selections and we will discuss at the end of this section.

Then we need the delay component. In the Boolean circuit, the delay is simply the data shifting following the clock, so it can be implemented with shift registers. But the register in the MTNCL pipeline can't do that. The design of the register and completion detection need to be changed a little bit with the pattern delay functionality. For the register, replacing the th12m gates in Fig.2 with resettable th12nm or th12dm gates, it can be changed to reset to 'NULL' or reset to 'DATA' registers, as the regnm and regdm shown in Fig.12. For the comp, replacing the th22 gate in the last stage in Fig.3 with th22d gates, the Ko can be changed to requesting for NULL (rfn) after reset. Removing the combination logic from the MTNCL pipeline, we can get the data pattern shift register as shown in Fig. 12.

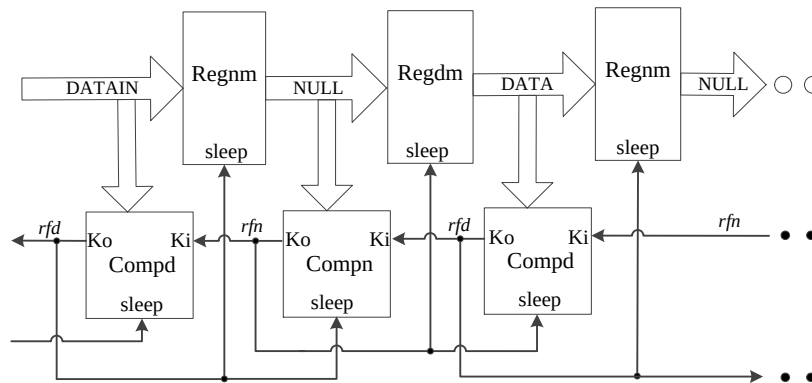


Fig.12. Data pattern shift register in MTNCL

Now you have all the three components in the FIR structure, put them together you will have a tap-generic FIR filter with fixed 8-bit input. The structure is shown in Fig.13. There are two pipelines in this architecture, the bottom one convolutes the input data and the top one shifts the input data. It looks good and produces correct result. But the throughput is very bad.

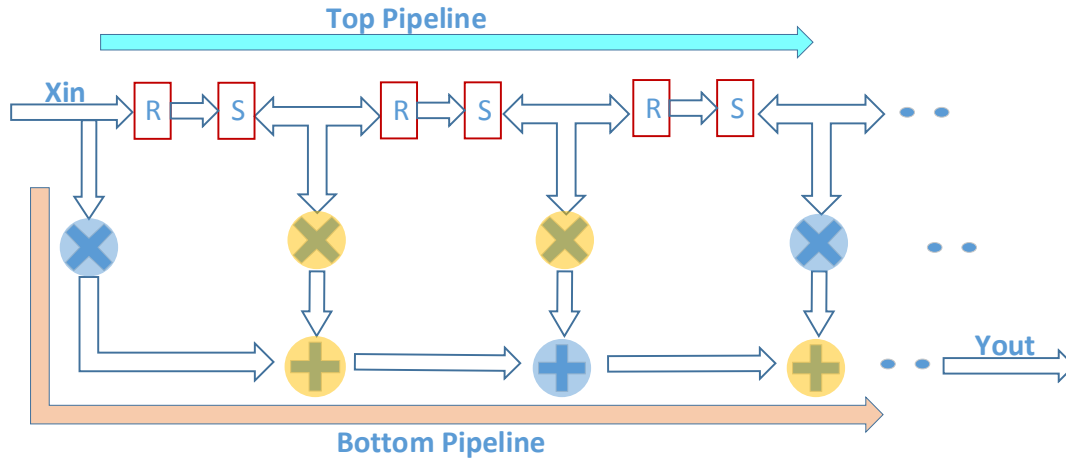


Fig.13. Generic FIR Filter Implementation in MTNCL

Now we will analyze this design. For the two pipelines architecture, after reset, the data path in the bottom one are all in 'NULL' cycle. While the data path in the top pipeline is reset to "DATA" and "NULL" patterns for it was designed as the pattern delay shift register. You can take the bottom pipeline as 'empty' and the top pipeline as already 'full' after reset. The DATA can propagate through an 'empty' pipeline but need to extrude a DATA to enter a 'full' pipeline. So when the first external data comes into the pipelines, it propagates through the bottom pipeline but is stuck at the first register in the top pipeline. After propagation delay of the bottom pipeline, which is named as the latency in a pipeline circuit, the top pipeline can move forward and those two pipelines will be able to take in next data. So the throughput of this architecture is the reciprocal of the latency, not the max Tdd in the pipeline stages.

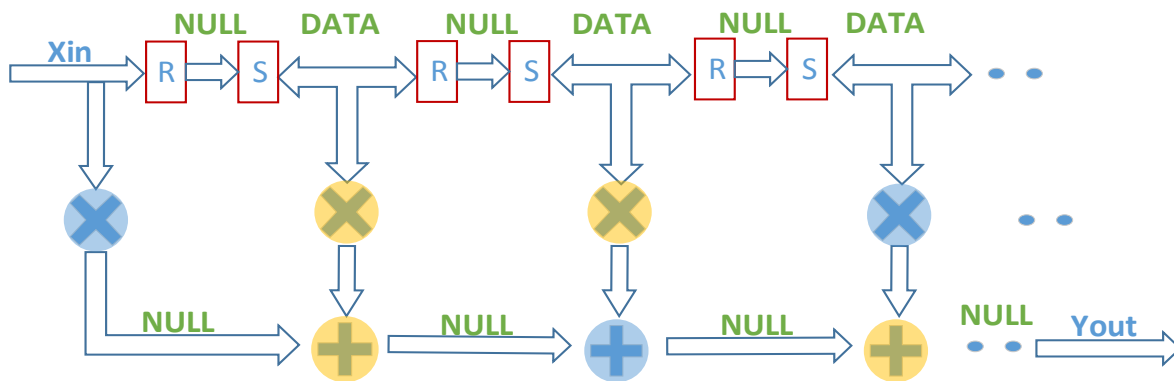


Fig.14. Data Path Status of the FIR Filter after Reset

There are multiple ways to fix it. One method is adding multiple stages in the top pipeline, as shown in Fig.15. So after reset, the top pipeline has the same number of 'NULL' cycles as the bottom one, then the DATA in the top pipeline can move forward shortly after internal data comes in.

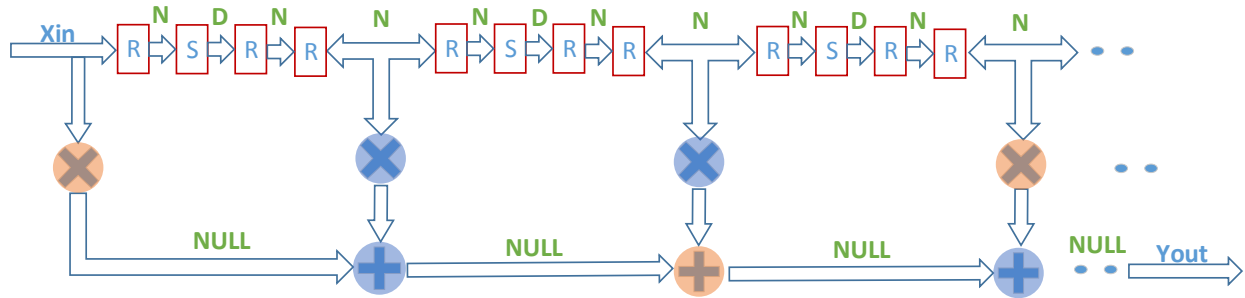


Fig.15. Generic FIR Filter with Optimized Throughput

The design is much better now. Let's go back to the two issues with the multiplier and adder selection at the beginning of this section. The first question is, will the multiplier and adder have the same throughput if the combinational delay is equal in MTNCL pipeline? The answer is negative if you looked through section 2 and section 3. And the multiplier has worse throughput than the adder. The second issue for this design is the adders. In the first tap of the FIR, the input bits to the adder are 16 by 16. While in the second tap, the input bits to the adder will be 16 by 17. Following the data flow in the bottom pipeline, the number of input bits to the adder increase. So the throughput for this design will deteriorate as it scales up. Then how to fix? The second one should be much easier than the first one.

5. Latency in the MTNCL pipeline

Throughput and latency are the two criterions in the pipeline. We have discussed a lot of the throughput optimization. Now let's look at the latency. In Boolean pipeline, the throughput is the reciprocal of the clock period, and the latency is the pipeline stages multiple the clock period.

$$\text{Throughput} = 1 / \text{clock_period}$$

$$\text{Latency} = N * \text{clock_period}$$

The clock period should be larger than the propagation delay between any pipeline stages. So both the throughput and latency are in worst-case in Boolean pipeline. You can decrease the latency by reducing the number N, but that will deteriorate the throughput. In the NCL or MTNCL pipeline, the throughput is constrained by the maximum Tdd, which can be estimated as the worse-case stage delay of any stage in the pipeline. So NCL/MTNCL also have the worst case performance in the pipelined architecture. But the latency is different. Think about you put one single data at the input of the MTNCL pipeline. It will propagate through all the NULL cycles in an "empty" pipeline before the data appears at the output. So the latency is not in the worst case as in the Boolean. It is the actually delay of the critical path in the circuit.

$$\text{Throughput} = 1 / \text{Max}(\text{Tdd})$$

$$\text{Latency} = \sum \text{Tdd}$$

Take a look at the three different designs in Fig.16. R is the register, and C is the combination logic. If there is no combination logic between the registers, the pipeline is a FIFO buffer. So for the first design, the buffer is placed ahead of the combination units. And in the second one, the buffer is placed after the combination units. The third design breaks the buffer and sandwiches the combination units. In terms of latency and throughput, they have no difference in Boolean. In MTNCL, the three have the same latency;

and after the latency time, they also have the same throughput. But it is a little tricky before the first data pops out. Which one do you think have the best throughput in the latency time? You know the answer if you see why we use Fig.15 for the FIR throughput optimization. Then think about what kind of circuit you can build with the latency benefit in the MTNCL pipeline—something beyond the Boolean design. That's why we need the asynchronous circuit.

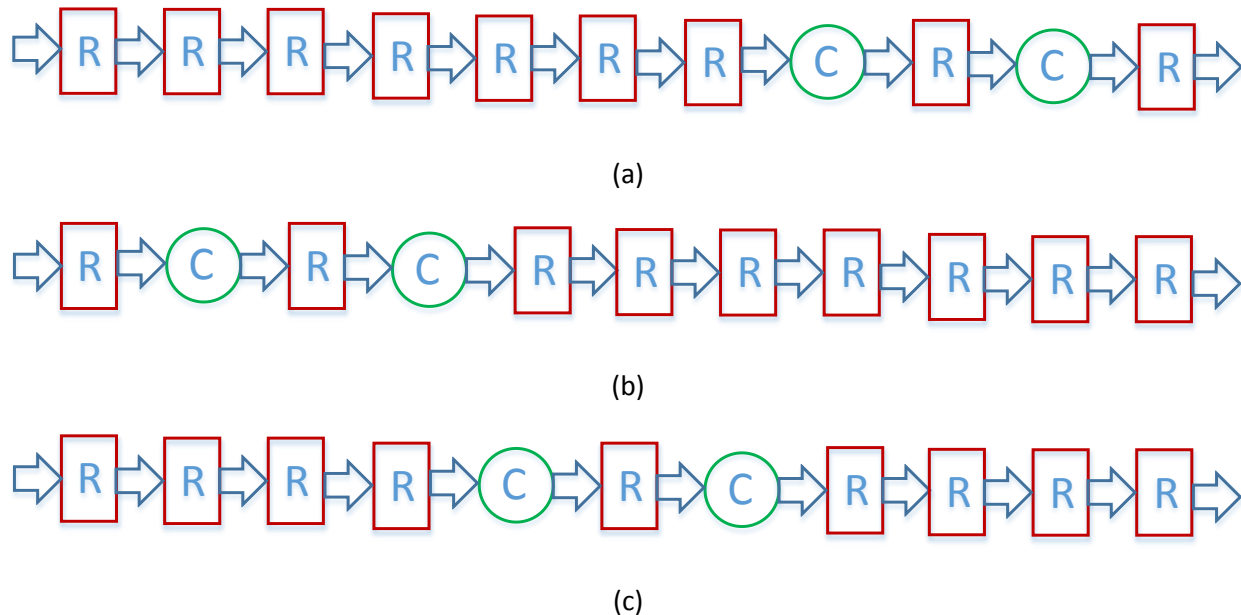


Fig. 16. MTNCL Pipeline with FIFO Buffers

6. Reference

- [1] S. C. Smith, "Gate and Throughput Optimizations for NULL Convention Self-Timed Digital Circuits," Ph.D. Dissertation, School of Electrical Engineering and Computer Science, University of Central Florida, May 2001.
- [2] Multipliers & Pipelining. <http://web.mit.edu/6.111/www/f2008/handouts/L09.pdf>
- [3] NCL Throughput Optimization. http://www.ndsu.edu/pubweb/~scotsmit/NCL_opt.pdf.
- [4] <http://www.ndsu.edu/pubweb/~scotsmit/about.html>
- [5] Parviz Palangpour, "CAD Tools for Synthesis of Sleep Convention Logic," Ph.D. Dissertation, Department of Electrical Engineering, University of Arkansas, May 2013.

Appendix I

VHDL Code for Generic Ripple Carry Adder in MTNCL

```
Library IEEE;

use IEEE.std_logic_1164.all;
use work.ncl_signals.all;

entity addw is
    generic(width: in integer := 8);
    port(x: in dual_rail_logic_vector(width-1 downto 0);
        y: in dual_rail_logic_vector(width-1 downto 0);
        ki, rst, sleepin: in std_logic;
        p: out dual_rail_logic_vector(width downto 0);
        sleepout: out std_logic;
        ko: out std_logic);
end;

architecture arch of addw is
    component FAM
        port(CIN, X, Y: in dual_rail_logic;
            sleep: in std_logic;
            COUT, S: out dual_rail_logic);
    end component;

    component HAM
        port(X, Y: in dual_rail_logic;
            sleep: in std_logic;
            COUT, S: out dual_rail_logic);
    end component;

    component regm is
        port(a: in dual_rail_logic;
            sleep: in std_logic;
            z: out dual_rail_logic );
    end component;

    component compm is
        generic(width: in integer := 4);
        port(a: IN dual_rail_logic_vector(width-1 downto 0);
```

```

        ki, rst, sleep: in std_logic;
        ko: OUT std_logic);
end component;
component th22x0 is
    port(a: in std_logic;
        b: in std_logic;
        z: out std_logic );
end component;
signal ppt: dual_rail_logic_vector(2*width-1 downto 0);
signal reglout : dual_rail_logic_vector(2*width-1 downto 0);
signal reg2in : dual_rail_logic_vector(2*width-1 downto 0);
signal kor1, kor2, korla, korlb : std_logic;
begin
Row:  for i in 0 to width-1 generate
    ppt(i) <= x(i);
    ppt(i + width) <= y(i);
end generate;
Gens0reg:  for i in 0 to 2*width-1 generate
Gs0reg: regm
    port map(ppt(i), kor1, reglout(i));
end generate;
Gr0acomp: compm
    generic map(width)
    port map(x, kor2, rst, sleepin, korla);
Gr0bcomp: compm
    generic map(width)
    port map(y, kor2, rst, sleepin, korlb);
THin0: th22x0
port map (korla, korlb, rst, kor1);
GFA0: HAm
    port map(reglout(width), reglout(0), kor1, reg2in(1), reg2in(0));
GenFA:  for i in 1 to width-1 generate
GFA: FAm

```

```

        port map(reglout(i), reglout(i+width), reg2in(2*i-1), kor1, reg2in(2*i+1),
reg2in(2*i));

        end generate;

Genslreg:   for i in 0 to width-1 generate

    Gslreg: regm

        port map(reg2in(2*i), kor2, P(i));

end generate;

    Gswreg: regm

port map(reg2in(2*width-1), kor2, P(width));

    Grlcomp: compm

        generic map(2*width)

        port map(reg2in, ki, rst, kor1, kor2);

    ko <= kor1;

    sleepout <= kor2;

end arch;

```


Appendix II

VHDL Code for Generic Carry Save Multiplier in MTNCL

```
Library IEEE;
use IEEE.std_logic_1164.all;
use work.ncl_signals.all;
entity multw is
    generic(width: in integer := 8);
    port(x: in dual_rail_logic_vector(width-1 downto 0);
        y: in dual_rail_logic_vector(width-1 downto 0);
        ki, rst, sleepin: in std_logic;
        p: out dual_rail_logic_vector(2*width-1 downto 0);
        sleepout: out std_logic;
        ko: out std_logic);
end;
architecture arch of multw is
    component FAm
        port(CIN, X, Y: in dual_rail_logic;
            sleep: in std_logic;
            COUT, S: out dual_rail_logic);
    end component;
    component CSAm is
        port (
            AI : IN dual_rail_logic ;
            BI: IN dual_rail_logic ;
            CIN: IN dual_rail_logic ;
            SIN: IN dual_rail_logic;
            sleep: in std_logic;
            COUT: OUT dual_rail_logic ;
            SOUT: OUT dual_rail_logic );--*
    end component ;
    component regm is
        port(a: in dual_rail_logic;
            sleep:in std_logic;
            z: out dual_rail_logic );
```

```

end component;

component bufm is
    port(a: in dual_rail_logic;
          sleep: in std_logic;
          z: out dual_rail_logic );
end component;

component genregm is
    generic(width: in integer := 4);
    port(a: IN dual_rail_logic_vector(width-1 downto 0);
          sleep: in std_logic;
          z: out dual_rail_logic_vector(width-1 downto 0));
end component;

component compm is
    generic(width: in integer := 4);
    port(a: IN dual_rail_logic_vector(width-1 downto 0);
          ki, rst, sleep: in std_logic;
          ko: OUT std_logic);
end component;

type Ctype is array (width+1 downto 0) of dual_rail_logic_vector(width downto 0);
signal a, b: dual_rail_logic_vector(width-1 downto 0);
signal creg, sreg: Ctype;
signal reg2in, reg2out : dual_rail_logic_vector(2*width-1 downto 0);
signal kor1, kor2: std_logic;
signal sigdata0: dual_rail_logic;

begin
sigdata0.rail1 <= '0';
sigdata0.rail0 <= '1';

Ggnd: for i in 0 to width-1 generate
    creg(0)(i) <= sigdata0;
    sreg(0)(i) <= sigdata0;
    sreg(i)(width) <= sigdata0;
end generate;

    creg(0)(width) <= sigdata0;

```

```

    sreg(0)(width) <= sigdata0;
Gens0reg:    for i in 0 to width-1 generate
    Gs0reg: regm
        port map(x(i), kor1, a(i));
    Gs0buf: bufm    --Y is considered as the coeffiency, so it is always DATA, use bufm
to generate the NULL; otherwise change bufm to regm.
        port map (y(i), kor1, b(i));
end generate;
Grlcomp: compm
    generic map(width)
        port map(x, kor2, rst, sleepin, kor1);

--logic
GcsaX: for i in 0 to width-1 generate
    GcsaY: for j in 0 to width-1 generate
        Gcsa: CSAm
            port map(a(j), b(i), creg(i)(j), sreg(i)(j+1), kor1, creg(i+1)(j),
sreg(i+1)(j));
        end generate;
    end generate ;
    creg(width+1)(0) <= sigdata0;
    sreg(width)(width) <= sigdata0;
Gfam: for i in 0 to width-1 generate
    Gfa: FAm
        port map(creg(width+1)(i), creg(width)(i), sreg(width)(i+1), kor1,
creg(width+1)(i+1), sreg(width+1)(i));
        end generate;
Gmreg: for i in 0 to width generate
    reg2in(i) <= sreg(i+1)(0);
end generate;
Gmreg2: for i in width+1 to 2*width-1 generate
    reg2in(i) <= sreg(width+1)(i-width);
end generate;
-- end of logic

```

```

Genslreg:    for i in 0 to 2*width-1 generate
  Gslreg: regm
    port map(reg2in(i), kor2, reg2out(i));
end generate;

G2comp: compm
  generic map(2*width)
    port map(reg2in, ki, rst, kor1, kor2);

  ko <= kor1;
  sleepout <= kor2;
  p <= reg2out;
end arch;

```

Appendix III

VHDL Code for 8-bit Input Generic FIR Filter in MTNCL

```
Library IEEE;

use IEEE.std_logic_1164.all;
use work.ncl_signals.all;
use work.functions.all;
use work.MTNCL_gates.all;
use work.fir_pack.all;

entity firn is
    generic(tap: in integer := 3);
    port(
        Xin: in dual_rail_logic_vector(7 downto 0);
        ki, rst, sleepin: in std_logic;
        Pout: out dual_rail_logic_vector(15+tap downto 0);
        sleepout: out std_logic;
        ko: out std_logic);
end firn;

architecture arch of firn is
    component multw is
        generic(width: in integer := 8);
        port(x: in dual_rail_logic_vector(width-1 downto 0);
            y: in dual_rail_logic_vector(width-1 downto 0);
            ki, rst, sleepin: in std_logic;
            p: out dual_rail_logic_vector(2*width-1 downto 0);
            sleepout: out std_logic;
            ko: out std_logic);
    end component;

    component addw is
        generic(width: in integer := 8);
        port( x: in dual_rail_logic_vector(width-1 downto 0);
            y: in dual_rail_logic_vector(width-1 downto 0);
            ki, rst, sleepin: in std_logic;
            p: out dual_rail_logic_vector(width downto 0);
```

```

        sleepout: out std_logic;
        ko: out std_logic);
end component;
component th22nx0 is
    port(a: in std_logic;
        b: in std_logic;
        rst: in std_logic;
        z: out std_logic );
end component;
component th22dx0 is
    port(a: in std_logic;
        b: in std_logic;
        rst: in std_logic;
        z: out std_logic );
end component;
component ShiftRegMTNCL is
    generic(width: in integer := 4;
        value: in bit_vector := "0110");
    port(wrapin : in dual_rail_logic_vector(width-1 downto 0);
        ki : in std_logic;
        rst : in std_logic;
        sleep : in std_logic;
        wrapout: out dual_rail_logic_vector(width-1 downto 0);
        sleepout: out std_logic;
        ko: out std_logic);
end component;
type Xtype is array (1 to tap) of DUAL_RAIL_LOGIC_VECTOR(7 downto 0);
type Htype is array (0 to tap) of DUAL_RAIL_LOGIC_VECTOR(7 downto 0);
type Ytype is array (1 to tap) of DUAL_RAIL_LOGIC_VECTOR(tap+14 downto 0);
type dXtype is array (1 to tap) of STD_LOGIC_VECTOR(7 downto 0);
type dHtype is array (0 to tap) of STD_LOGIC_VECTOR(7 downto 0);
type dYtype is array (1 to tap) of STD_LOGIC_VECTOR(tap+14 downto 0);
signal X: Xtype;
signal H: Htype;

```

```

signal A: Ytype;
signal B: Ytype;
signal sleepax: std_logic_vector(tap downto 1);
signal koa: std_logic_vector(tap downto 1);
signal sleepxr: std_logic_vector(tap downto 1);
signal sleepa: std_logic_vector(tap downto 1);
signal sleepx: std_logic_vector(tap downto 1);
signal kor: std_logic_vector(tap-1 downto 0);
signal kox: std_logic_vector(tap downto 0);
signal koxr: std_logic_vector(tap-1 downto 0);
signal H_q, H_qbar : std_logic_vector((tap+1)*8 - 1 downto 0);
begin
    ADDERTap: addw
        generic map(15+tap)
            port map(A(tap)(14+tap downto 0), B(tap)(14+tap downto 0), ki, rst,
sleepax(tap), Pout, sleepout, koa(tap));

    GenAdder: for i in tap-1 downto 1 generate
        ADDER: addw
            generic map(15+i)
                port map(A(i)(14+i downto 0), B(i)(14+i downto 0), koa(i+1), rst,
sleepax(i), A(i+1)(15+i downto 0), sleepa(i+1), koa(i));
        end generate GenAdder;

    GenMult: for i in tap downto 1 generate
        Mult: multw
            generic map (8)
                port map(X(i), H(i), koa(i), rst, sleepxr(i), B(i)(15 downto 0), sleepx(i),
kox(i));
        end generate GenMult;

        Mult0: multw
            generic map (8)
                port map(Xin, H(0), koa(1), rst, sleepin, A(1)(15 downto 0), sleepa(1),
kox(0));

        ShiftReg0: ShiftRegMTNCL
            generic map(8, "00000000")

```

```

        port map(Xin, koxr(1), rst, sleepin, X(1), sleep(1), kor(0));
GenReg: for i in tap-2 downto 1 generate
    ShiftReg: ShiftRegMTNCL
        generic map(8, "00000000")
        port map(X(i), koxr(i+1), rst, sleep(1), X(i+1), sleep(i+1), kor(i));
end generate GenReg;
    ShiftReglast: ShiftRegMTNCL
        generic map(8, "00000000")
        port map(X(tap-1), kox(tap), rst, sleep(tap-1), X(tap), sleep(tap),
kor(tap-1));
    THngate: for i in tap downto 1 generate
        THout: th22nx0
            port map (sleepa(i), sleepx(i), rst, sleepax(i));
    end generate THngate;
    THdgate: for i in tap-1 downto 0 generate
        THout: th22dx0
            port map (kor(i), kox(i), rst, koxr(i));
    end generate THdgate;
    ClearB1:    for m in 1 to tap generate
        ClearB2:    for n in 16 to 14+tap generate
            B(m)(n).raill <= '0';
            B(m)(n).rail0 <= '1';
        end generate ClearB2;
    end generate ClearB1;
    ko <= koxr(0);
    ReadinH: for i in 0 to tap generate
        H(i) <= Int_to_DR(Tarray(i), 8);
    end generate;
end arch;

-----

--Pattern-delay shift-register in MTNCL

-----

library ieee;
use ieee.std_logic_1164.all;

```



```

use ieee.numeric_std.all;
use work.ncl_signals.all;
use work.functions.all;

entity ShiftRegMTNCL is
    generic(width: in integer := 4;
           value: in bit_vector := "0110");
    port(wrapin : in dual_rail_logic_vector(width-1 downto 0);
         ki : in std_logic;
         rst : in std_logic;
         sleep : in std_logic;
         wrapout: out dual_rail_logic_vector(width-1 downto 0);
         sleepout: out std_logic;
         ko: out std_logic);
end ShiftRegMTNCL;

architecture arch of ShiftRegMTNCL is
    component genregrstm is
        generic(width: in integer := 4;
               dn : in bit := '1';
               value: in bit_vector := "0110");
        port(a: IN dual_rail_logic_vector(width-1 downto 0);
             rst : in std_logic;
             sleep: in std_logic;
             z: out dual_rail_logic_vector(width-1 downto 0));
    end component;
    component compm is
        generic(width: in integer := 4);
        port(a: IN dual_rail_logic_vector(width-1 downto 0);
             ki, rst, sleep: in std_logic;
             ko: OUT std_logic);
    end component;

    component compdm is

```

```

        generic(width: in integer := 4);
        port(a: IN dual_rail_logic_vector(width-1 downto 0);
            ki, rst, sleep: in std_logic;
            ko: OUT std_logic);
    end component;
component Eregm is
    generic(width: in integer := 8);
    port(x: in dual_rail_logic_vector(width-1 downto 0);
        ki, rst, sleep: in std_logic;
        y: out dual_rail_logic_vector(width-1 downto 0);
        sleepout: out std_logic;
        ko: out std_logic);
end component;
    signal wrap, r12, wrapbuf: dual_rail_logic_vector(width-1 downto 0);
    signal c1, c2, kibuf: std_logic;
begin
    Gregdata: genregrstm
        generic map(width, '1', value)           ----reset to DATA
        port map(wrapin, rst, c1, r12);
    Gcompnull: compm                               -----reset to request for NULL
        generic map(width)
        port map(wrapin, c2, rst, sleep, c1);
    Gregnull: genregrstm
        generic map(width, '0', value)           --reset to NULL
        port map(r12, rst, c2, wrap);
    Gcompdata: compdm
        generic map(width)
        port map(r12, kibuf, rst, c1, c2);         --reset to request for DATA
    Gbuf: Eregm
        generic map(width)
        port map(wrap, ki, rst, c2, wrapbuf, sleepout, kibuf);
    wrapout <= wrapbuf;
    ko <= c1;
end arch;

```